

Project Handoff: Agent Reliability Platform & Interactive AI Video

Context for an agent tasked with creating GitHub issues to scaffold both projects

June 2026

Contents

0. Your Mission (read this first)	2
1. Scrubless Today — Technical Inventory	3
1.1 Repository structure	3
1.2 Tech stack	4
1.3 Data model (current)	4
1.4 ML pipeline (existing — this is the reusable engine)	4
1.5 What’s production-grade vs experimental	5
1.6 Team context	5
2. Project A — Agent Reliability Platform (“ARP”)	5
2.1 Product vision (high-level)	5
2.2 System architecture (high-level)	6
2.3 Component breakdown (low-level)	7
2.3.1 SDK (the customer’s installed dependency)	7
2.3.2 Ingest API	7
2.3.3 Realtime scorer	7
2.3.4 Eval system	7
2.3.5 Recovery policies	8
2.3.6 Web dashboard	8
2.4 Scrubless reuse map	8
2.5 MVP scope (in / out for YC application)	9
2.6 Work breakdown (organized so you can create issues directly)	9
Epic: SDK Foundation (Python)	9
Epic: Ingest API	9
Epic: Real-time scoring	10
Epic: Web dashboard	10
Epic: Docs + landing	10
Epic: Dogfood + design partners	10
Epic: YC application prep	10
Decisions to surface as decision:needs-human	11
2.7 Effort estimate (rolled up)	11
2.8 Suggested milestones	11
2.9 Dependency graph (epic-level)	11
3. Project B — Interactive AI Video Experiences (“IAV”)	11
3.1 Product vision (high-level)	11

3.2 System architecture (high-level)	12
3.3 Component breakdown (low-level)	13
3.3.1 Extended ingest pipeline	13
3.3.2 Persona agent backend	13
3.3.3 Real-time voice/video response	13
3.3.4 Branch state machine	13
3.3.5 Authoring UI (creator side)	13
3.3.6 Viewer UI	13
3.4 Scrubless reuse map	14
3.5 MVP scope (in / out for YC application)	14
3.6 Work breakdown (organized for issue creation)	14
Epic: Extended ingest pipeline	14
Epic: Persona agent backend	15
Epic: Voice + lip-sync response	15
Epic: Web player + viewer UX	15
Epic: Authoring UI	15
Epic: Demo content	15
Epic: Landing + marketing	16
Epic: YC application prep	16
Decisions to surface as decision:needs-human	16
3.7 Effort estimate (rolled up)	16
3.8 Suggested milestones	16
3.9 Dependency graph (epic-level)	17
4. Comparative Reuse Matrix	17
5. Team Capacity Assumptions	17
6. GitHub Issue Creation Guidelines	18
6.1 Repository setup	18
6.2 Label taxonomy	18
6.3 Milestones	18
6.4 Issue template	18
6.5 Sizing convention	19
6.6 Issue creation order	19
6.7 What “good” looks like	19
6.8 What to NOT create issues for	19
7. Decision Points You Cannot Make	19
8. Sanity Checks Before You Start	20
9. Closing Note	20

0. Your Mission (read this first)

You are a fresh agent inheriting context. Your job is to create well-scoped GitHub issues that let a 3-engineer team execute on **either** of two candidate pivots from the existing Scrubless codebase. The team has NOT yet committed to one — they want to see both projects broken down concretely before deciding. After they pick one, the other’s issues will be closed in bulk.

Deliverable expectations:

1. Create issues for **both projects** (tagged `project:arp` or `project:iav`).

2. Each issue should be one engineer-day to one engineer-week of work. Bigger units belong in kind:epic and get split into child issues.
3. Use the milestone, label, and template conventions in Section 7 of this document.
4. For each project, ensure issue ordering forms a valid execution sequence (dependencies surfaced via “Blocked by #X” / “Blocks #Y”).
5. Surface decisions you cannot make yourself as decision:needs-human issues — do not silently choose architectural directions for the team.
6. Skip issues for anything in the “Out of MVP scope” subsections — those exist for the team’s reference, not your backlog.

Where things live: - Repo: getscrubless/clipfind (current Scrubless monorepo — both new projects will be implemented as sibling top-level directories or new repos; that’s a decision:needs-human issue you should surface, not decide) - Existing docs to read: docs/ARCHITECTURE.md, docs/PIVOT_FINALISTS.md, docs/PIVOT_ANALYSIS.md - Existing CLAUDE.md: /CLAUDE.md (project conventions)

1. Scrubless Today — Technical Inventory

This is what exists. You’ll reference these modules constantly when writing reuse-tagged issues.

1.1 Repository structure

```
clipfind/
├── app.py                # FastAPI entry; mounts routers; auth/session middleware
├── access.py            # Anon-resource permissioning, require_video/collection
├── admin.py             # /api/admin/stats + /admin page; geo aggregates
├── auth.py              # Auth0 OAuth, current_user, is_admin
├── billing.py           # Stripe checkout, portal, tier upgrades
├── config.py            # All env vars; TIER_LIMITS; ROOT/STORAGE paths
├── countries.py         # ISO-2 → name/flag + SVG path id mapping
├── embeddings.py        # CLIP model load, describe_frame (Claude Vision),
                        #   embed_text, ChromaDB `segments` collection
├── geo.py               # request_country() – CDN header reader
├── indexing.py          # process_video (frames+transcript+embed),
                        #   transcript_context, restore_state, ingest_sample
├── library.py           # /api/library/* – folder upload + scan + index
├── models.py            # SQLAlchemy: User, Video, Collection, Event;
                        #   record_event(); ensure_columns() migration;
                        #   persist_video / persist_collection
├── pages.py            # Marketing pages (/blog, /pricing, /about);
                        #   _HEAD/_FOOT shell; post markdown rendering
├── ratelimit.py         # slowapi limiter (X-Forwarded-For keyed)
├── reels.py            # /api/reel – ffmpeg concat pipeline for highlight reels
├── search.py            # /api/search/* + /api/qa/* (single video + library)
├── state.py             # In-memory VIDEOS / COLLECTIONS / SAMPLE_ID / REELS
├── usage.py             # Monthly Q&A + hours-indexed cap enforcement
├── videos.py           # /api/upload, /api/status, /api/videos/*/source
├── index.html           # Single-file SPA (vanilla JS, no build step)
├── posts/*.md           # Blog markdown
└── scrubby/            # Static assets (CSS, SVGs, world.svg, logos)
```

```
└─ storage/          # Uploaded videos + extracted frames (gitignored)
└─ docs/             # ARCHITECTURE.md, PITCH.md, render.sh
```

1.2 Tech stack

Layer	Choice	Notes
Web framework	FastAPI (single process)	Async, type hints used throughout
Vector store	ChromaDB (in-process, persistent)	Collection named segments; metadata: video_id, collection_id, timestamp, frame_path, transcript_segment
ORM	SQLAlchemy 2.x (Mapped types)	SQLite local; Postgres on Railway
Visual encoder	OpenCLIP ViT-B/32	Loaded into Python process at startup
Transcription	OpenAI Whisper API	Word-level timestamps; chunked to 15-30s segments
Vision enrichment	Anthropic Claude Vision	claude-sonnet-4-6; top 3 results enriched
Q&A	Anthropic Claude	Sonnet-4-6, 600 max tokens, transcript-grounded
Auth	Auth0 (oauthlib)	Session cookies via starlette SessionMiddleware
Billing Frontend	Stripe (subscriptions + portal) Vanilla JS + CSS, single index.html	Webhook-driven tier updates No build step. scrubby/scrubless.css carries the design tokens.
Deployment	Railway (single VPS-style box)	uvicorn --proxy-headers --forwarded-allow-ips=* Keyed off X-Forwarded-For
Rate limiting	slowapi	

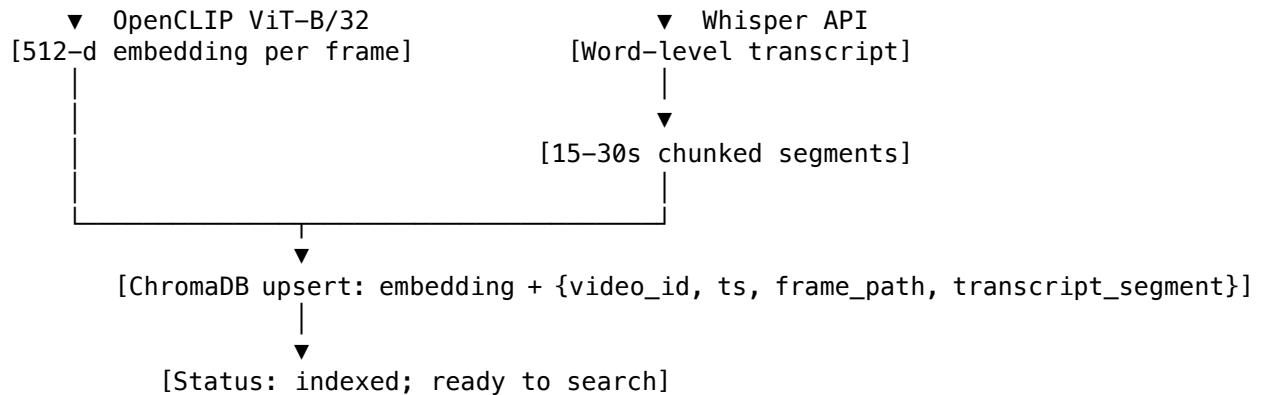
1.3 Data model (current)

```
users      (id, auth0_sub, email, tier, stripe_customer_id,
           stripe_subscription_id, created_at, country)
videos     (id, collection_id, title, source, status, total_segments,
           owner, created, summary, chapters)
collections (id, name, path, created, owner)
events      (id, kind, at, source, user_id, country)
            -- kind ∈ {view, upload, search, qa, reel}
            -- country: ISO-2 from CDN header, NULL if unknown
```

Plus ChromaDB segments collection storing CLIP embeddings + metadata per frame.

1.4 ML pipeline (existing — this is the reusable engine)





At query time: - Text query → CLIP text embedding → ChromaDB cosine search → top N - Top 3 enriched with Claude Vision description - Ask: relevant segments → context → Claude prompt with citation instruction → cited answer

1.5 What's production-grade vs experimental

Production-grade (depend on it): - The ingest pipeline (ffmpeg → CLIP → Whisper → ChromaDB) - Auth0 session handling - Stripe billing + cap enforcement - Admin dashboard analytics + geo aggregation - The reels.py ffmpeg cut pipeline (proven by live use)

Experimental / will need hardening for a real product: - Single-process FastAPI (no horizontal scaling story) - ChromaDB in-process (fine to a few million embeddings; will need swap for scale) - No multi-tenant isolation at the row level (videos belong to users but not enforced at ChromaDB layer) - No async job system for long-running work (background tasks via FastAPI BackgroundTasks; not durable across restarts)

The Scrubless engine is suitable as a **starting point** for either pivot but neither MVP ships as-is. Both pivots need new scaffolding (multi-tenant + durable jobs + production observability).

1.6 Team context

3 senior engineers: - Salesforce (enterprise CRM, APIs, integrations) - GEICO (insurance, call-center ops, regulated industry) - LinkedIn (web-scale infra: ClickHouse, Kafka, distributed systems)

No GTM specialist. All-engineering team — issues should NOT include sales-call scripts, partner outreach lists, etc. (Those exist but live in docs/ not in GitHub.)

Velocity assumption: ~50 productive engineer-hours/week across the team if not all full-time on the project, ~120/week if all full-time. Size issues with the lower assumption.

2. Project A — Agent Reliability Platform (“ARP”)

2.1 Product vision (high-level)

The production reliability layer for AI agents — Datadog × Sentry × Braintrust, but agent-native. Four primitives, in order of importance:

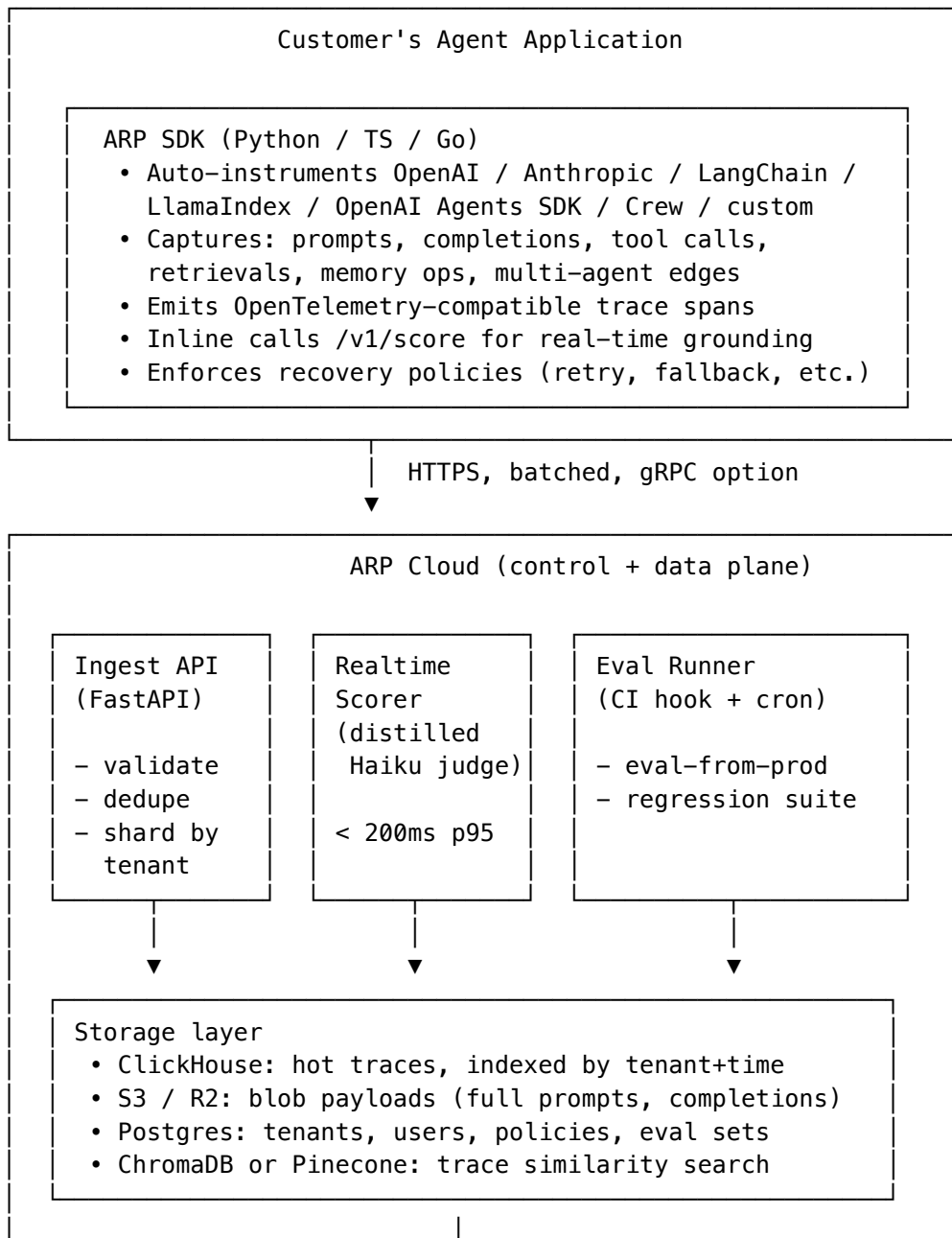
1. **Trace observability.** Capture every agent run as a graph of LLM calls + tool calls + memory ops + retrievals. Render it so a developer can debug *why* an agent did what it did in < 30 seconds.
2. **Real-time grounding + hallucination scoring.** Inline judge model scores every LLM output against retrieved context. Score is visible on the trace *before* response returns to the user.

3. **Production-derived eval.** Failed traces convert to regression tests in one click. CI hook runs the suite on every deploy.
4. **Recovery policies.** Declarative YAML: “if grounding < 0.6 → retry with adjusted prompt”; “if tool error twice → escalate human.” The SDK acts on the policy without app-code changes.

Buyer: AI engineering teams (Series A-C agent startups + F500 internal AI teams). Self-serve developer purchase via OSS SDK → paid cloud.

Why now: Hundreds of agent companies hit production in 2025; reliability is now \$\$\$-blocking. Distilled judge models cleared sub-200ms latency. OpenTelemetry semantic conventions for LLM/AI ratified in early 2026.

2.2 System architecture (high-level)





Web Dashboard (Next.js)

- Trace list + multi-agent graph view (Sankey/DAG)
- Drill-down to per-call detail (prompt+completion)
- One-click "turn failed trace into eval"
- Eval suite runner + history
- Recovery policy editor (YAML)
- Cost attribution + alerting

2.3 Component breakdown (low-level)

2.3.1 SDK (the customer's installed dependency)

Python first. TS and Go are stretch.

Key modules: - `arp/instrumentation/openai.py` — patches `openai.chat.completions.create`, `openai.embeddings.create`, etc. - `arp/instrumentation/anthropic.py` — patches `anthropic.messages.create`, streaming variant - `arp/instrumentation/langchain.py` — LangChain-Tracer-style callback handler - `arp/instrumentation/llama_index.py` — instrument retriever + query pipeline - `arp/instrumentation/openai_agents.py` — OpenAI Agents SDK integration - `arp/trace.py` — Trace + Span dataclasses; OTEL-compatible serialization - `arp/client.py` — async HTTP client; batches + retries; backpressure - `arp/scoring.py` — inline call to `/v1/score`; async option - `arp/policies.py` — YAML policy parser + executor - `arp/context.py` — context manager for grouping spans into traces

Critical correctness requirements: - Zero-impact when ARP cloud is unreachable (degrade gracefully; never raise into customer app) - Sample rate configurable; default to 100% for first 30 days, then auto-throttle - PII scrubbing hooks (configurable redaction patterns)

2.3.2 Ingest API

- `POST /v1/traces` — batched trace ingestion (gzip body)
- `POST /v1/score` — sync grounding score for an LLM output
- `POST /v1/policies/check` — query the active policy for a given trace context
- Authn: API key in header (`x-arp-key`); tenant id derived from key
- Storage writes: ClickHouse (hot trace summary), S3 (full payload blobs)

2.3.3 Realtime scorer

- Distilled judge model (target: Haiku-class, 200-300ms p95)
- Score axes: grounding (0-1), instruction adherence (0-1), safety (0-1)
- Cache scoring by (`prompt_hash`, `context_hash`) to skip duplicate calls
- Fallback: if scorer is overloaded, return `null` score, let SDK proceed unblocked

2.3.4 Eval system

- `eval_sets` table: name, owner, created
- `eval_cases` table: input, expected, `eval_set_id`, `source_trace_id` (nullable)
- `eval_runs` table: `eval_set_id`, version, `pass_rate`, ts
- CLI: `arp eval run <set>` returns pass/fail + diff vs prior run
- GitHub Action: PR comment with pass-rate delta

2.3.5 Recovery policies

YAML schema (illustrative):

```
policies:
- name: retry_on_low_grounding
  when:
    grounding_score: { lt: 0.6 }
  action:
    kind: retry
    with:
      extra_context: similar_traces(k=3)
      max_attempts: 2
- name: fallback_to_larger_model
  when:
    retry_attempt: { gte: 2 }
  action:
    kind: switch_model
    to: anthropic/claude-opus-4-8
```

The SDK reads the customer's policies on init and enforces them locally — no roundtrip per decision.

2.3.6 Web dashboard

Sections: - **Traces** — list view with filters (tenant, kind, score, time range) - **Trace detail** — graph view of multi-agent run; click a node  drawer with prompt/completion - **Evals** — list of suites, run history, pass-rate chart - **Policies** — YAML editor + live preview against last N traces - **Settings** — API keys, billing, integrations, team

Stack: Next.js + Tailwind + shadcn/ui + Tanstack Query. ClickHouse queries via a thin Python BFF.

2.4 Scrubless reuse map

Scrubless piece	Reuse for ARP?	Notes
FastAPI scaffolding	Yes	Patterns from <code>app.py</code> + routers carry over
<code>ensure_columns()</code> migration	Yes	Copy-paste idea for additive schema changes
Multi-tenant <code>project_id</code> pattern (designed for Stripe-for-Video)	Yes	Apply to traces table
Auth0 session + admin gating	Partial	Use Auth0 patterns; ARP needs API-key auth too (new)
Stripe billing + portal	Yes	Reuse for ARP self-serve checkout
ChromaDB for similarity search	Maybe	ARP needs trace similarity (“find traces like this failed one”); could reuse Chroma
Anthropic client for judge model	Yes	Same anthropic SDK pattern as in <code>search.py</code>
Admin dashboard time-windowed analytics	Yes	Adapt the windowed-query pattern from <code>admin.py</code> for trace analytics
slowapi rate limiting	Yes	Apply per API key instead of per IP

Scrubless piece	Reuse for ARP?	Notes
<code>record_event()</code> pattern	Conceptually	ARP's whole product IS this pattern, generalized
ffmpeg pipeline / CLIP / Whisper	No	Not needed; ARP is text/JSON only
<code>pages.py</code> marketing shell	Partial	Reusable for ARP landing page if same repo
<code>index.html</code> SPA	No	ARP dashboard is Next.js (or similar SSR)

Reuse-percentage estimate: ~25-35%. Mostly *patterns* (auth, migrations, multi-tenant, billing) rather than code (most of Scrubless's value is in the video ML pipeline, which ARP doesn't use). Honest framing: Scrubless is the *case study* that proves you can build agent systems; ARP is a fresh codebase that borrows Scrubless's *engineering patterns* and uses Scrubless as the first dogfooding customer.

2.5 MVP scope (in / out for YC application)

In scope (must ship in 6 weeks): - Python SDK auto-instrumenting OpenAI + Anthropic + LangChain - Ingest API + ClickHouse trace storage + S3 blob storage - Web dashboard: trace list + multi-agent graph view + per-call drill-down - Real-time grounding scorer (distilled Haiku judge, sub-300ms p95) - Eval-from-prod: one-click "make this an eval"; manual run, no CI hook yet - Landing page + docs site (Mintlify/Fern) with quickstart - 3-5 design partners signed (cold outreach, week 5) - Scrubless instrumented as dogfood case study

Out of MVP scope (V2): - TS / Go SDKs - Recovery policies (V1 has manual retry; V2 has declarative policies) - CI hook (GitHub Action for eval-on-PR) - Multi-region storage - SOC2 / on-prem - Cost attribution per business outcome - Advanced multi-agent visualizations (V1 has basic Sankey) - Adversarial robustness testing (prompt-injection sims)

2.6 Work breakdown (organized so you can create issues directly)

Each row below should map to ~1 issue. Effort: S = ≤1 day, M = 2-4 days, L = ≤1 week, XL = needs splitting (you should split before creating).

Epic: SDK Foundation (Python)

- [S] Set up `arp/` package skeleton + dev tooling (pytest, ruff, mypy)
- [M] Define Trace + Span dataclasses; OTel-compatible serialization
- [M] Build async batching HTTP client with retries + backpressure
- [M] Implement context manager API for grouping spans into traces
- [M] OpenAI instrumentation (chat completions + embeddings + streaming)
- [M] Anthropic instrumentation (messages + streaming)
- [L] LangChain callback-handler integration + smoke tests
- [S] Configuration loading (env vars + programmatic init)
- [S] Graceful degradation when cloud unreachable
- [M] PII scrubbing hooks (configurable regex + custom callbacks)
- [S] SDK README + quickstart

Epic: Ingest API

- [S] FastAPI project skeleton (reuse Scrubless patterns)
- [M] POST `/v1/traces` endpoint with schema validation + gzip handling

- [M] Tenant/project model (tenants, api_keys tables)
- [M] API-key authn middleware
- [L] ClickHouse setup + trace schema + write path
- [M] S3-compatible blob storage for full payloads
- [S] Health + metrics endpoints
- [S] Per-key rate limiting (adapt slowapi from Scrubless)
- [M] Trace dedup + idempotency keys
- [M] Background worker to compact + tier ClickHouse → S3 cold

Epic: Real-time scoring

- [L] Distilled judge model selection + benchmark (Haiku-3.5 baseline; measure latency)
- [M] POST /v1/score endpoint
- [M] Score caching layer keyed on (prompt_hash, context_hash)
- [S] Async option in SDK (don't block customer response)
- [M] Score axes definition (grounding, instruction adherence, safety)
- [L] Calibrate score thresholds against curated test set

Epic: Web dashboard

- [S] Next.js project setup with Tailwind + shadcn
- [M] Auth integration (NextAuth via Auth0)
- [M] Trace list view with filters
- [L] Trace detail page with multi-agent graph (react-flow library)
- [M] Per-call drill-down drawer (prompt + completion + scores)
- [M] Eval suites list + detail page
- [M] "Turn this trace into eval" one-click action
- [M] Eval run + history view
- [S] Settings: API keys, team, billing
- [L] Billing integration (reuse Scrubless Stripe patterns)
- [S] Onboarding flow (first key, first install, first trace)

Epic: Docs + landing

- [S] Mintlify (or Fern) site skeleton
- [M] Quickstart guide (install → first trace in 5 minutes)
- [M] Per-framework integration guides (OpenAI / Anthropic / LangChain)
- [S] API reference (auto-generated from OpenAPI)
- [M] Marketing landing page
- [S] "Why ARP" comparison page (vs LangSmith, Helicone, Datadog LLM)

Epic: Dogfood + design partners

- [L] Instrument Scrubless's own agent endpoints with ARP SDK
- [M] Pull last 30 days of Scrubless traces + render in dashboard as the demo dataset
- [M] Cold-DM script + tracking spreadsheet for 30 agent-startup CTOs
- [L] Onboard first 3 design partners (10+ hours each)

Epic: YC application prep

- [S] Founder bios + team page
- [M] Founder video (1 min)
- [M] Application narrative draft
- [S] Submit by 2026-07-27

Decisions to surface as decision: needs-human

- Same repo or new repo for ARP code?
- ClickHouse hosted (ClickHouse Cloud / Tinybird) vs self-hosted?
- Auth provider: stick with Auth0 (consistent with Scrubless) or move to Clerk/WorkOS?
- Pricing tier exact dollar values
- Open-source license for SDK (Apache 2.0 recommended)
- Cloud region(s) for V1 (single us-east vs multi-region from day 1)

2.7 Effort estimate (rolled up)

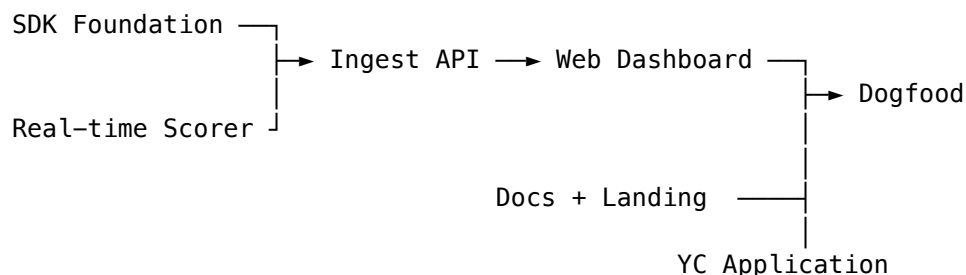
Epic	Engineer-hours	Calendar with 3 eng (~120 h/wk total)
SDK Foundation	80-100	1 week
Ingest API	80-100	1 week
Real-time scoring	50-70	0.5-1 week
Web dashboard	120-160	1.5-2 weeks
Docs + landing	40-60	0.5 week
Dogfood + design partners	60-80	parallel, ~1 week of focused calls
YC application	20-30	parallel
Total MVP	450-600	5-6 weeks parallelized

Achievable for 2026-07-27 if all three engineers go heads-down full-time starting now. Tight but real.

2.8 Suggested milestones

- m1-foundation (weeks 1-2): SDK + Ingest API + ClickHouse
- m2-product (weeks 3-4): Dashboard + Realtime scoring + Eval-from-prod
- m3-launch (weeks 5-6): Docs, landing, dogfood, design partners, YC app

2.9 Dependency graph (epic-level)



3. Project B — Interactive AI Video Experiences (“IAV”)

3.1 Product vision (high-level)

Every video on the internet today is passive: you watch, you stop. IAV turns video into a stateful, conversational, branchable surface: - **Pause and ask the video a question** — the on-screen person turns toward you and answers, voice and lip-sync matching their real persona. - **Branch the narrative** — “show me

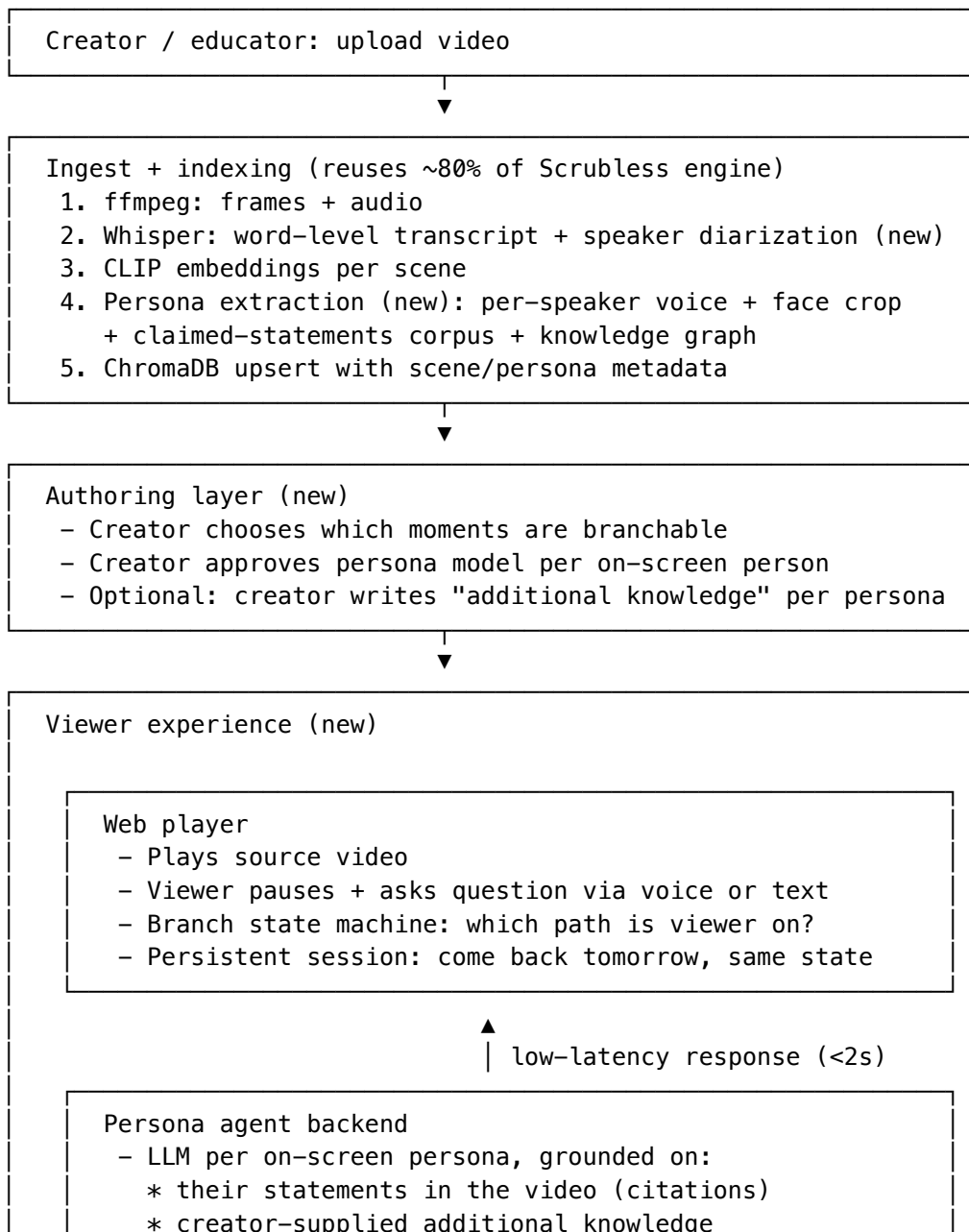
this scene from the other character's POV", "skip ahead to the part about X", "what would have happened if Y?" - **Personalize** — only show the moments relevant to my role / situation / level.

Buyer (initial wedge — pick one): - Education / training (Khan Academy + corporate L&D) - Sales demo content (interactive product demos) - Onboarding / customer success videos

The Pivot Finalists doc recommends **education / training as the V1 wedge** because (a) creators retain rights, (b) ICP has budget, (c) interactivity solves a real engagement problem.

Why now: Voice cloning + lip-sync hit cinema quality in 2025 (Cartesia, ElevenLabs, Synthesia-class). Sub-2s real-time multimodal model responses are viable (GPT-5-realtime). a16z thesis #6 names this directly.

3.2 System architecture (high-level)



- * persona-style system prompt
- Voice cloned from their actual audio (ElevenLabs API)
- Lip-sync via Synthesia/Tavus API or self-hosted model
- Output: video clip of persona answering, spliced in

3.3 Component breakdown (low-level)

3.3.1 Extended ingest pipeline

Beyond Scrubless's existing ingest: - **Speaker diarization.** WhisperX or pyannote-audio. Output: per-segment speaker labels. - **Face detection + tracking.** RetinaFace or MediaPipe. Output: per-frame face bounding boxes + identity clusters. - **Speaker-to-face linking.** Heuristic + small classifier (audio activity + lip movement). Output: speaker ID linked to face cluster. - **Voice fingerprinting.** Resemblyzer or similar. Output: voice embedding per speaker. - **Persona corpus extraction.** For each speaker, collect all their statements as a corpus indexed in ChromaDB with speaker_id metadata.

3.3.2 Persona agent backend

- personas table: id, video_id (or corpus_id), name, voice_id, face_thumbnail, style_prompt
- Per-persona system prompt template grounding the agent on the persona's actual statements + style
- Persona-aware retriever: filter ChromaDB by speaker_id so the persona only "knows" what they actually said
- Optional creator-supplied additional knowledge merged into context

3.3.3 Real-time voice/video response

- ElevenLabs voice clone API (production) or Cartesia (lower latency)
- Lip-sync: integration with Tavus / HeyGen API initially; consider self-hosted SyncNet/Wav2Lip later for cost
- Spliced playback: pre-rendered intro/outro clips + dynamically generated response clip
- Response clip caching: same question often asked twice; cache the rendered video

3.3.4 Branch state machine

- sessions table: viewer_id, video_id, current_node_id, history (JSON array of moments visited)
- Branch graph: stored per video as JSON; each node has child links + transition conditions
- Branch UI: timeline shows current path; viewer can rewind to a fork and choose differently
- Persistent state: session resumes across devices via viewer_id (anon or signed-in)

3.3.5 Authoring UI (creator side)

- Upload video → automatic persona extraction
- Creator reviews + approves each detected persona (renames, adds bio, supplies additional context)
- Creator marks moments as "branchable" with a quick gesture in the player
- Preview mode: creator experiences their own video as a viewer would

3.3.6 Viewer UI

- Standard video player + "Ask" button overlay
- Voice input (Web Speech API + Cartesia fallback) or text input
- Response renders as overlaid video clip (persona speaking)
- Branch markers visible on the timeline

- Session sync via cookie + optional account

3.4 Scrubless reuse map

Scrubless piece	Reuse for IAV?	Notes
ffmpeg frame + audio extraction	Yes (100%)	Use as-is
Whisper transcription	Yes	Add diarization wrapper
CLIP embeddings	Yes (100%)	Per-scene; same code
ChromaDB retrieval	Yes	Add speaker_id, scene_id metadata
Claude Vision enrichment	Yes	For scene descriptions; persona contexts
reels.py ffmpeg cut pipeline	Yes	Splice generated response clips into the player feed
Library mode (collection of videos)	Yes	A “course” = a collection; persona learned across all
FastAPI scaffolding + auth + billing	Yes	Reuse
record_event() analytics	Yes	Engagement events per viewer per session
Admin dashboard	Yes	Repurpose for creator analytics
Vanilla JS frontend pattern	Maybe	Player needs richer interactivity; consider React for viewer UI
Marketing pages shell	Yes	Reuse for IAV landing + docs

Reuse-percentage estimate: ~70-80%. IAV is a *much* more direct extension of Scrubless than ARP — most of the engine carries over verbatim, and the new work is at the edges (diarization, persona agents, voice cloning, lip-sync, branch state, viewer UX).

3.5 MVP scope (in / out for YC application)

In scope (must ship in 6-10 weeks): - Ingest pipeline extended with diarization + face detection + persona extraction - Persona agent backend with text-only Q&A (voice + lip-sync as week 7-8) - Web player with pause-and-ask (text input first, voice second) - Authoring UI: upload □ auto-extract personas □ creator approves - One polished demo video (e.g., interactive version of a well-known lecture or tutorial) - Landing page + docs

Stretch for YC application (if time permits): - Voice cloning + lip-sync response (initially via Tavus/HeyGen API) - Branch state machine with at least one branchable demo

Out of MVP scope (V2): - Self-hosted lip-sync model - Mobile app - Real-time multi-viewer experiences - Marketplace of interactive videos - Enterprise SSO + analytics dashboards - Multi-language (V1 is English-only)

Honest note: IAV’s MVP demo bar is high — Tavus/HeyGen/Synthesia have set a polished baseline. Shipping a *credible* interactive-video demo in 6 weeks is genuinely hard. 10 weeks is realistic; 8 weeks is a stretch; 6 weeks requires cutting voice/lip-sync from the YC demo.

3.6 Work breakdown (organized for issue creation)

Epic: Extended ingest pipeline

- [M] Add WhisperX (or pyannote) for speaker diarization

- [M] Integrate face detection (RetinaFace or MediaPipe) + tracking
- [L] Speaker-to-face linking (audio activity + lip-sync heuristic)
- [M] Voice fingerprinting per speaker (Resemblyzer or similar)
- [M] Persona corpus extraction → ChromaDB with speaker_id metadata
- [S] Migration: add personas table + relations to videos
- [M] Background job system for the extended pipeline (heavier than Scrubless's current; consider Celery or Dramatiq)

Epic: Persona agent backend

- [M] Define Persona model + serializer
- [M] Per-persona retriever (filter by speaker_id)
- [L] Persona-aware system prompt builder (grounding on actual statements + style)
- [M] Optional creator-supplied additional context (merged at query time)
- [M] API endpoint: POST /api/personas/{id}/ask → text answer with citations
- [S] Citation rendering: timestamps that link back to source moments

Epic: Voice + lip-sync response

- [L] ElevenLabs voice cloning integration (per persona)
- [L] Cartesia (or alternative) for lower-latency response generation
- [L] Tavus / HeyGen lip-sync API integration
- [M] Response clip rendering pipeline (audio + lip-sync video)
- [M] Response clip caching layer (key on question hash + persona id)
- [S] Cost monitoring / rate limiting per persona

Epic: Web player + viewer UX

- [M] React (or Lit) component for video player with overlay UI
- [M] Pause-and-ask button + text input
- [L] Voice input integration (Web Speech API + Cartesia fallback)
- [M] Spliced playback: switch between source clip and response clip
- [L] Branch state machine: track viewer path, persist via session
- [M] Branch timeline UI: visualize current path, allow rewind to fork
- [S] Viewer session API: anonymous + optional account

Epic: Authoring UI

- [M] Creator dashboard: video list + processing status
- [L] Persona review + approval flow (rename, add bio, additional context)
- [L] Branch authoring: mark moments as forks, define child nodes
- [M] Preview mode: experience as viewer
- [S] Publish flow: generate shareable URL

Epic: Demo content

- [L] Pick + secure rights to one excellent source video (recommend: a famous public-domain lecture or a sponsored demo with a willing creator)
- [M] Author the interactive experience end-to-end (review personas, define branches, write supplementary context)
- [M] QA across desktop + mobile, test 50+ viewer questions

Epic: Landing + marketing

- [S] Reuse marketing shell from Scrubless
- [M] Landing page with embed of the demo
- [S] Docs for creators (how to author)
- [M] Outreach to 30 creators (educators, YouTubers, L&D leads) for V2 design partners

Epic: YC application prep

- [S] Founder bios
- [M] Founder video (1 min) featuring the live demo
- [M] Application narrative draft

Decisions to surface as decision: needs-human

- Same repo or new repo (more likely new for IAV — fundamentally different product)?
- Lip-sync: API (Tavus/HeyGen, fast/expensive) vs self-hosted (slower setup, cheaper at scale)?
- Voice provider: ElevenLabs (quality) vs Cartesia (latency)?
- Job orchestrator: Celery vs Dramatiq vs Temporal? (Scrubless uses FastAPI BackgroundTasks; IAV needs more)
- Player tech: React (industry standard, larger surface area) vs Lit (smaller, faster)?
- Demo content: public-domain lecture (cheap, low engagement) vs sponsored creator (expensive, viral potential)?
- Pricing model: per-creator subscription vs per-minute-of-video-watched vs revenue-share with creator?

3.7 Effort estimate (rolled up)

Epic	Engineer-hours	Calendar with 3 eng
Extended ingest pipeline	80-110	1 week
Persona agent backend	60-80	0.5-1 week
Voice + lip-sync response	100-140	1.5 weeks
Web player + viewer UX	130-170	1.5-2 weeks
Authoring UI	80-110	1 week
Demo content	60-90	1 week (heavy on judgment, not coding)
Landing + marketing	40-60	0.5 week
YC application	20-30	parallel
Total MVP	570-790	7-10 weeks parallelized

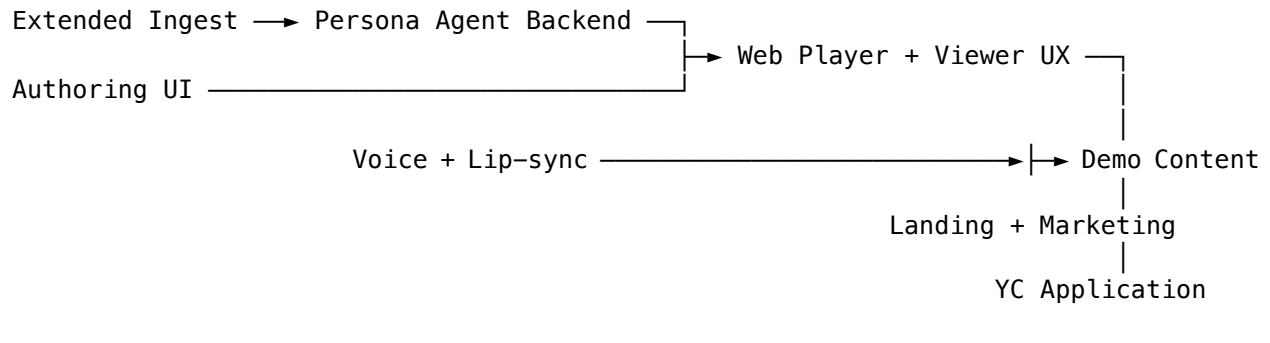
This does not fit in 6 weeks unless voice/lip-sync is cut from the YC demo. The text-only version (pause-and-ask with persona giving a text answer + citations to source moments) IS shippable in 6 weeks.

3.8 Suggested milestones

- m1-engine (weeks 1-3): Extended ingest + persona agent backend
- m2-experience (weeks 4-6): Web player + authoring UI + text-only Q&A demo
- m3-multimedia (weeks 7-9): Voice + lip-sync (post-YC if needed)
- m4-launch (week 10): Demo content + landing + YC application

If team must ship by 2026-07-27, scope cut: skip m3-multimedia entirely; YC demo is text-Q&A version.

3.9 Dependency graph (epic-level)



4. Comparative Reuse Matrix

Scrubless component	ARP reuse	IAV reuse
ffmpeg + frame extraction	No	Yes (100%)
OpenCLIP + embeddings	No	Yes (100%)
Whisper transcription	No	Yes + diarization wrapper
ChromaDB vector store	Maybe (trace similarity)	Yes
Claude Vision enrichment	No	Yes
reels.py ffmpeg pipeline	No	Yes (spliced playback)
FastAPI scaffolding	Yes	Yes
SQLAlchemy + ensure_columns	Yes	Yes
Auth0 patterns	Yes	Yes
Stripe billing	Yes	Yes
slowapi rate limiting	Yes (per-API-key)	Yes (per-tenant)
Marketing pages shell	Partial	Yes
Vanilla JS SPA	No	Partial
Admin dashboard analytics	Pattern only	Pattern only
record_event analytics	Generalize	Yes
Estimated reuse	25-35%	70-80%

The pattern is clear: - **ARP** is a *new product class* that borrows Scrubless's engineering patterns (auth, billing, multi-tenant, migrations, analytics) but builds new ML pipelines and data planes. - **IAV** is a *natural extension* of Scrubless's engine — most of the existing code carries forward, and the new work is around the edges (diarization, persona agents, voice/lip-sync, viewer UX).

This affects velocity: IAV starts with more done; ARP starts more from scratch but has cleaner separation of concerns.

5. Team Capacity Assumptions

- 3 engineers, all senior
- Stack familiarity: Python/FastAPI (all), TS/React (varies), ClickHouse/Kafka (LinkedIn engineer), distributed systems (LinkedIn engineer)
- Velocity: 120 productive engineer-hours/week if all full-time; 50/week if part-time
- No GTM, no design, no PM specialist — issues should not assume these roles exist

- Outreach + cold-DM work is shared overhead, ~10 hours/week split across the team
-

6. GitHub Issue Creation Guidelines

6.1 Repository setup

- One repo per project, named arp and iav respectively (recommend new repos; decision:needs-human issue should ask the team to confirm)
- Existing clipfind repo stays as Scrubless production
- Each new repo: main branch protected; PRs require 1 reviewer; CI must pass

6.2 Label taxonomy

project:arp	project:iav	
kind:epic	kind:feature	kind:bug
kind:chore	kind:docs	kind:decision
priority:p0	priority:p1	priority:p2
size:s (≤1d)	size:m (2-4d)	size:l (≤1w)
area:sdk	area:backend	area:dashboard
area:dx	area:infra	area:ml
status:blocked	status:ready	status:in-progress
decision:needs-human	dogfood	design-partner

6.3 Milestones

For ARP: - m1-foundation (target 2026-07-04) - m2-product (target 2026-07-18) - m3-launch (target 2026-07-27)

For IAV: - m1-engine (target 2026-07-11) - m2-experience (target 2026-07-25) - m3-multimedia (target 2026-08-15, post-YC) - m4-launch (target 2026-07-27 for text-only YC demo)

6.4 Issue template

```
## What
[1-2 sentences describing the work]

## Why
[Why this matters; what user/customer outcome it serves]

## Acceptance criteria
- [ ] Specific testable outcome 1
- [ ] Specific testable outcome 2
- [ ] Tests added (unit / integration / e2e as appropriate)
- [ ] Docs updated if user-facing

## Implementation notes
[Architectural pointers, files to touch, libraries to use, gotchas]

## Reuse from Scrubless
[Specific Scrubless modules/patterns to reference; or "N/A - new code"]

## Dependencies
```

Blocked by: #N
Blocks: #M

6.5 Sizing convention

- `size:s` — ≤ 1 engineer-day; trivial PR, low review burden
- `size:m` — 2-4 engineer-days; standard PR
- `size:l` — ≤ 1 engineer-week; large PR or multiple coordinated PRs
- Anything bigger $\square$ split into child issues under an `kind:epic`

6.6 Issue creation order

1. First pass: create all `kind:epic` issues per project (top-level structure)
2. Second pass: create per-epic child `kind:feature` issues, linked to the epic
3. Third pass: create `kind:decision` issues for everything in the “Decisions to surface” sections
4. Fourth pass: assign milestones + dependencies (Blocked by / Blocks)
5. Final pass: review the resulting dependency graph for cycles; surface anything illogical as a comment

6.7 What “good” looks like

A well-formed issue: - Title is a verb phrase (“Add diarization to ingest pipeline”, “Build OpenAI auto-instrumentation”) - Acceptance criteria are *testable* (avoid “make it work well”) - Implementation notes name specific files or libraries (not “use a library for X”) - Reuse-from-Scrubless field is specific (not just “see Scrubless”) - Size label is accurate (better to underestimate than overestimate) - Linked to its epic + milestone

6.8 What to NOT create issues for

- Anything in an “Out of MVP scope” list above
 - GTM activities (sales emails, partnership outreach) — these are tracked elsewhere
 - Marketing copy (only “create landing page” as a structural issue, not “write the hero headline”)
 - Decisions where you have a clear architectural opinion that doesn’t conflict with the team’s stated patterns — make the decision in the issue, don’t surface as `decision:needs-human`
 - Anything obviously redundant with another issue (check for dupes)
-

7. Decision Points You Cannot Make

These genuinely need human input. Create `kind:decision` issues for each:

Universal: - Same monorepo or split into separate repos? - Which project does the team commit to (after seeing both backlogs)? - Pricing model + tier dollar values - Hiring plan post-YC

ARP-specific: - ClickHouse hosted (ClickHouse Cloud / Tinybird) vs self-managed? - Auth provider (Auth0 / Clerk / WorkOS) - TS / Go SDK order of operations after Python ships - Open-source license (Apache 2.0 recommended)

IAV-specific: - Lip-sync: API (Tavus, HeyGen) vs self-hosted - Voice provider: ElevenLabs (quality) vs Cartesia (latency) - Player UI tech: React vs Lit vs Vue - V1 content strategy: public-domain demo vs paid creator partnership - Pricing model: creator subscription vs per-view vs revenue-share

8. Sanity Checks Before You Start

Before creating issues, verify:

1. You can read docs/PIVOT_FINALISTS.md and it matches this doc's framing
2. You can read docs/ARCHITECTURE.md and understand current Scrubless module layout
3. The GitHub repos arp and iav either exist or have a kind:decision issue asking the team to create them
4. You have permissions to create issues, labels, and milestones on those repos
5. The team has the ~600 engineer-hours of capacity per project assumed in Section 2.7 / 3.7

If any of these fail, surface the gap as a kind:decision issue or message to the team — do not silently work around it.

9. Closing Note

The hardest engineering work has already been done — Scrubless is a real, shipped multimodal agent system. Both candidate pivots leverage that foundation, but in opposite directions:

- **ARP** reuses ~30% of Scrubless code but ~80% of Scrubless's *credibility* as the founder-narrative case study
- **IAV** reuses ~80% of Scrubless code but creates a fundamentally new consumer-facing product

The team's decision is more about *what they want to live in for 5+ years* than about technical feasibility. Both ship. Both can win YC. Build out both backlogs cleanly so the team can choose based on the texture of what shipping each actually looks like — then close the loser in bulk.

Your job is to make that decision *easy* by surfacing the work clearly. Resist the temptation to optimize: better to create slightly more issues than to merge things that should be separate. The team can close noise; they can't see work that was never written down.

Good luck.